

But when I told him what had happened, Bezos looked horrified. He did not say “I’m so sorry.” He did not say “Do you need anything?” Instead, he made a face, and in an instant, an aide came and whisked him away. When presented with the opportunity for empathy, even performative empathy, he chose escape.

A few hours later, on the private plane home, a famous movie producer offered my wife a blanket. My children’s faces were covered in spots. Under my fingernails, red welts were beginning to rise.

The world has always been run by rich men. The robber barons of the Gilded Age were known for their ruthlessness in the accumulation of wealth—hiring Pinkertons to shoot striking unionists. But they directly engaged with the world around them, using their wealth and power to muscle it into its most profitable form. And although today’s billionaires are clearly manipulating society to maximize their own profit, something else is also happening—a disassociation from the reality of cause and effect, from meaning and history. These men no longer feel the need to change the world in order to succeed, because their success is guaranteed, no matter what happens to the rest of us.

“I’m finished,” yells Daniel Plainview, perched happily on the polished floor of his own celestial kingdom. Though he has just committed a crime, he has never felt so free.

This article appears in the May 2026 print edition with the headline “Everything Is Free and Nothing Matters.”

FRED TALKS

31st May 2026

CONTENTS

Everything You Know About Latency Is Wrong

TYLER TREAT · 2691 WORDS

Please for the love of all that is holy: Stop writing long boring
Titles

SWYX · 1431 WORDS

What I Learned About Billionaires at Jeff Bezos’s Private Retreat

NOAH HAWLEY · 2436 WORDS

Everything You Know About Latency Is Wrong

TYLER TREAT · 01 MAR 2026 · [SOURCE](#)

Okay, maybe not *everything* you know about latency is wrong. But now that I have your attention, we can talk about why the tools and methodologies you use to measure and reason about latency are likely horribly flawed. In fact, they’re not just flawed, they’re probably *lying to your face*.

When I went to [Strange Loop](#) in September, I attended a workshop called “Understanding Latency and Application Responsiveness” by Gil Tene. Gil is the CTO of Azul Systems, which is most renowned for its C4 pauseless garbage collector and associated Zing Java runtime. While the workshop was four and a half hours long, Gil also gave a 40-minute talk called “[How NOT to Measure Latency](#)” which was basically an abbreviated, less interactive version of the workshop. If you ever get the opportunity to see Gil speak or attend his workshop, I recommend you do. At the very least, do yourself a favor and watch one of his recorded talks or find his slide decks online.

The remainder of this post is primarily a summarization of that talk. You may not get anything out of it that you wouldn’t get out of the talk, but I think it can be helpful to absorb some of these ideas in written form. Plus, for my own benefit, writing about them helps solidify it in my head.

What is Latency?

Latency is defined as **the time it took one operation to happen**. This means every operation has its own latency—with one million operations there are one million latencies. As a result, latency *cannot* be measured as *work units / time*. What we’re interested in is how latency *behaves*. To do this meaningfully, we must describe the complete distribution of latencies. Latency almost *never* follows a normal, Gaussian, or Poisson distribution, so looking at averages, medians, and even standard deviations is useless.

Latency tends to be heavily multi-modal, and part of this is attributed to “hiccups” in response time. Hiccups resemble periodic freezes and can be due to any number of reasons—GC pauses, hypervisor pauses, context switches, interrupts, database reindexing, cache buffer flushes to disk, etc. These hiccups never resemble normal distributions and the shift between modes is often rapid and eclectic.

with the air of a man who believed that nothing matters—poverty, chaos, [human suffering](#). He was having fun. It didn’t even matter that the [entire destructive exercise](#) ultimately yielded no practical financial gains. For him, the outcome was a foregone conclusion: He could only win, because losing had lost its meaning.

Since the 2024 election, there has been a philosophical shift on the right, and especially among tech billionaires, to vilify the idea of empathy. Musk has called empathy “the fundamental weakness of Western civilization.” He sees it as a weapon wielded by liberal society to bludgeon otherwise rational people into operating against their own interests. Empathy is something done to you by others—a vulnerability they exploit, a back door through which they gain access to your resources and will. This [rejection of empathy](#) as a human value gives cover to people who don’t want to feel anything at all. If empathy is the problem, then lack of it isn’t a deficiency—it’s [an advantage](#).

[Elizabeth Bruenig: The conservative attack on empathy](#)

I finally met Bezos on the last day of Campfire, at lunch, after my wife had broken her wrist. I went over to thank him for having us, and he asked how our Campfire experience had been. I told him that it was great, but that unfortunately my wife had broken her wrist that morning when she slipped on the wet grass while kicking a ball with our 6-year-old son.

The night before, we’d all stood by the pool at the beach club watching a cadre of synchronized swimmers execute a flawless water routine. I had spoken with a famous novelist, who said, “I just don’t understand why I’m here.” A famous rock star was about to start an acoustic set. The famous chef had made paella. Somewhere deep under my skin, a brutal pox was beginning to form.

The next morning, my wife fell, and I found myself in a black SUV with a team of private-security contractors, who whisked us to the back entrance of a Santa Barbara emergency room, where she was seen and treated right away. We made it back in time to watch the Supreme Court justice Zoom in from Washington, D.C.

How was your Campfire? Bezos asked me an hour later, and because I am an honest person, and because I have been a host myself, I decided he would want to know that there had been a problem, but that his team had reacted quickly and been extremely helpful. To be clear, I was in no way blaming him, nor was I shaking down the richest man on Earth. Instead, I was simply offering Bezos, also a husband and father, a brief human connection.

that his financial and social value could be affected by negative publicity. He still believed that his actions had consequences. He had not yet freed himself—the way Daniel Plainview freed himself—from the rules of men.

Eight years later, Bezos and two of the world’s other richest men—Mark Zuckerberg and Elon Musk—have clearly left the world of consequences behind. They float in a sensory-deprivation tank the size of the planet, in which their actions are only ever judged by themselves.

The closer I’ve gotten to the world of wealth, the more I understand that being truly rich doesn’t mean amassing enough money to afford superyachts, private jets, or a million acres of land. It means that everything becomes effectively free. Any asset can be acquired but nothing can ever be lost, because for soon-to-be trillionaires, no level of loss could significantly change their global standing or personal power. For them, the word *failure* has ceased to mean anything.

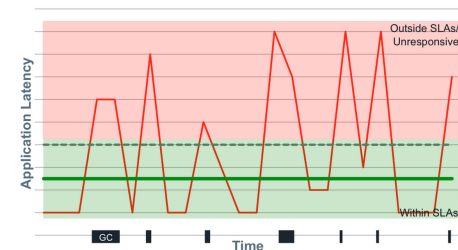
Adam Serwer: How America chose not to hold the powerful to account

This sense of invulnerability has deep psychological ramifications. If everything is free and nothing matters, then the world and other people exist only to be acted upon, if they are acknowledged at all. This is different from classic narcissism, in which a grandiose but fragile self-image can mask deep insecurity. What I’m talking about is a self-definition in which the individual grows to the size of the universe, and the universe vanishes. Asked recently if there is any check on his power, President Trump—himself a billionaire, and by far the richest president in American history—said, “Yeah, there is one thing. My own morality. My own mind. It’s the only thing that can stop me.” Not domestic or international law, not the will of the voters, not God or the centuries-old morality of civic and religious life.

Decades of research in developmental psychology have shown that moral reasoning develops through consequences—not punishment, necessarily, but experiencing the effects of your actions on others, receiving honest feedback, having to accommodate reality as it actually is rather than as you wish it to be. It’s not that the wealthy become evil; it’s that their environment stops teaching them the things that nonwealthy people are forced to learn simply by living in a world that pushes back. When you can buy your way out of any mistake, when you can fire anyone who disagrees with you, when your social circle consists entirely of people who need something from you, the basic mechanism by which humans learn that other people are real goes dark.

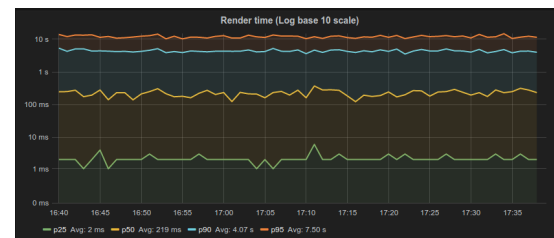
Thomas Chatterton Williams: The very powerful men who think introspection is dumb

When Peter Thiel said, “I no longer believe that freedom and democracy are compatible,” he wasn’t talking about your freedom. He was talking about his own. You don’t exist. When Musk took a chainsaw to the federal government as part of the inside joke he called DOGE, he did so



How do we meaningfully describe the distribution of latencies? We have to look at percentiles, but it’s even more nuanced than this. A trap that many people fall into is fixating on “the common case.” The problem with this is that there is a lot more to latency behavior than the common case. Not only that, but the “common” case is likely not as common as you think.

This is partly a tooling problem. Many of the tools we use do *not* do a good job of capturing and representing this data. For example, the majority of latency graphs produced by Grafana, such as the one below, are basically worthless. We like to look at pretty charts, and by plotting what’s convenient we get a nice colorful graph which is quite readable. Only looking at the 95th percentile is what you do when you want to hide all the bad stuff. As Gil describes, it’s a “marketing system.” Whether it’s the CTO, potential customers, or engineers—someone’s getting duped. Furthermore, *averaging* percentiles is mathematically absurd. To conserve space, we often keep the summaries and throw away the data, but the “average of the 95th percentile” is a meaningless statement. You cannot average percentiles, yet note the labels in most of your Grafana charts. Unfortunately, it only gets worse from here.



Gil says, “The number one indicator you should never get rid of is the maximum value. That is not noise, that is the signal. The rest of it is noise.” To this point, someone in the workshop naturally responded with “But what if the max is just something like a VM restarting? That doesn’t describe the behavior of the system. It’s just an unfortunate, unlikely occurrence.” By ignoring the maximum, you’re effectively saying “this doesn’t happen.” If you can identify the *cause* as noise, you’re okay, but if you’re not capturing that data, you have no idea of what’s *actually* happening.

How Many Nines?

But how many “nines” do I *really* need to look at? The 99th percentile, by definition, is the latency below which 99% of the observations may be found. Is the 99th percentile *rare*? If we have a single search engine node, a single key-value store node, a single database node, or a single CDN node, what is the chance we actually hit the 99th percentile?

Gil describes some real-world data he collected which shows how many of the web pages we go to actually experience the 99th percentile, displayed in table below. The second column counts the number of HTTP requests generated by a single access of the web page. The third column shows the likelihood of one access experiencing the 99th percentile. With the exception of google.com, every page has a probability of 50% or higher of seeing the 99th percentile.

Site	# of requests	page loads that would experience the 99thile (1 - (1 - 99% ^ reqs) ^ 1/999%)
amazon.com	180	85.2%
reddit.com	204	87.1%
ynet.com	112	87.8%
usafirmavenue.com	109	86.5%
...	...	...
nytimes.com	173	82.4%
cnn.com	278	83.9%
...	...	...
twitter.com	87	58.2%
ynetnews.com	86	57.0%
facebook.com	178	83.3%
...	...	...
google.com	71	29.7%
cnn.com that samples notice-free pages	70	53.4%
google.com search for "http requests per page"	70	53.4%

The point Gil makes is that the 99th percentile is what most of your web pages will see. It’s not “rare.”

What metric is more representative of user experience? We know it’s not the average or the median. 95th percentile? 99.9th percentile? Gil walks through a simple, hypothetical example: a typical user session involves five page loads, averaging 40 resources per page. How many users will *not* experience something *worse* than the 95th percentile? 0.003%. By looking at the 95th percentile, you’re looking at a number which is relevant to 0.003% of your users. This means 99.997% of your users are going to see *worse* than this number, so why are you even looking at it?

On the flip side, 18% of your users are going to experience a response time *worse* than the 99.9th percentile, meaning 82% of users will experience the 99.9th percentile or better. Going further, more than 95% of users will experience the 99.97th percentile and more than 99% of users will experience the 99.995th percentile.

The median is the number that 99.999999999% of response times will be *worse* than. This is why *median* latency is irrelevant. People often describe “typical” response time using a median, but the median just describes what everything will be worse than. It’s also the most commonly used metric.

That’s how the weekend started. Here’s how it ended: My wife broke her wrist slipping on wet grass, and both children and I came down with hand, foot, and mouth disease. This is not a joke. One of us went home with her arm in a sling; the other three developed itchy, painful red blisters all over our faces and extremities. If you’re looking for a sign from God as to whether hanging out with the richest man on Earth is right for you, pay attention when he sends you not one plague, but two. Suffice it to say we have never been back to Campfire, nor have we ever been invited.

At drinks on the second night, the head of a major talent agency asked me what I thought of the weekend. I said, “I’ve spent my whole career trying to figure out how the world works. I didn’t realize I could just come here and ask the people who ran it.” On some level I was kidding. The lead singer of an alt-country band didn’t run the world, nor did a noted author who would later be accused of impropriety. But finding myself at that resort by exclusive invitation, I now knew exactly what people meant when they talked about the elite.

Sitting in the lecture hall, pencils out, listening to a famous chef explain his humanitarian work, it was easy to feel like the solution to the world’s problems lay within our grasp. And yet, looking around at faces I had only ever seen in a magazine or on-screen, I had an unsettling revelation: This is the hubris of accomplishment. To be declared a genius at one thing is to begin to believe you are a genius at everything.

Here we were, 80 individuals with a combined net worth that was greater than a small city’s yet infinitesimal compared with the wealth and dominion of our host. How did he view this exercise—as a first step toward changing the world, or as a performative display of his reach and influence?

Bezos was everywhere that weekend—in a tight T-shirt, laughing too loudly, arms thrown around his teenage sons. He had recently become the world’s second centibillionaire, his net worth hovering somewhere around \$112 billion, about half of what it is today. That number, previously unimaginable, had made him unique on a planet of 8 billion people, and you could feel it in the room. Even the richest and most famous among us were drawn to the energy of this impossible wealth.

[Martin Baron: Where Jeff Bezos went wrong with *The Washington Post*](#)

Though we didn’t know it at the time, Bezos’s first marriage would be over a few weeks later. My defining impression of his wife that weekend was sadness, even though Bezos made a big show of performing the role of family man. In hindsight, it is that performance that sticks with me. The Jeff Bezos of 2018 acted as if he still believed that people’s impression of him mattered,

In 2018, I was a guest at Jeff Bezos’s Campfire retreat in Santa Barbara, California. It’s an annual event in which the Amazon founder invites 80-plus guests—celebrities, artists, intellectuals, and anyone else he thinks is interesting—to spend three nights at a private resort. I had recently been approached by Amazon about moving my film-and-television business over from Disney, and although I had declined (or maybe *because* I had declined), Bezos’s team invited me to Campfire, perhaps keen to impress me with the power of his reach.

From the March 2024 issue: The rise of techno-authoritarianism

On a warm October Thursday, a fleet of private jets was dispatched to airports in Van Nuys and New York to shepherd guests to Santa Barbara in style. At that point I had only a vague sense of who else was coming—famous people, rich people, influential people, and me. A guest list, I was told, would be given to us once we arrived. Families were invited; an on-site nanny would be provided *for each child*.

So my wife and I got our two children from Austin to Los Angeles and took a 45-minute jet ride north, with a television mogul and a comedian on board. Bezos had bought out the entire Biltmore resort for the weekend, as well as the beach club across the street. He had brought in a security firm from Las Vegas to ensure our safety and privacy. Even the weather felt expensive, and when we were shown to our rooms, the designer gift bags we found were filled with luxury goods.

Alexandra Petri: The 10 things the Bezoses are almost certainly grateful for each morning

Each morning, we gathered in a lecture hall to hear presentations. If you’ve ever seen a TED Talk, you understand the format. The year I went, a sitting Supreme Court justice was interviewed, and a neurologist talked about technological advances in prosthetics. In the afternoons and evenings, we were encouraged to exchange ideas over drinks and four-course meals, with no set purpose—to network, in other words, with some of the most rarefied talent on Earth. The most common question I heard was “Why am I here?”

“Why am I here?” asked the 1980s hair-metal singer. “Why am I here?” asked the Pulitzer Prize–winning novelist, the famous anthropologist, the presidential historian. Only the movie stars and the billionaires didn’t ask: They had done this kind of thing before. It turns out there is a circuit of idea festivals. Many tech billionaires host one, and if you find yourself on the right list, you can spend much of the year traveling the world, eating Wagyu, and discussing how to make the world a better place with the most famous talk-show host in history.

If it’s so critical that we look at a lot of nines (and it is), why do most monitoring systems *stop* at the 95th or 99th percentile? The answer is simply because “it’s hard!” The data collected by most monitoring systems is usually summarized in small, five or ten second windows. This, combined with the fact that we can’t average percentiles or derive five nines from a bunch of small samples of percentiles means there’s no way to know what the 99.999th percentile for the minute or hour was. We end up throwing away a lot of good data and losing fidelity.

A Coordinated Conspiracy

Benchmarking is hard. Almost all latency benchmarks are broken because almost all benchmarking tools are broken. The number one cause of problems in benchmarks is something called “coordinated omission,” which Gil refers to as “a conspiracy we’re all a part of” because it’s everywhere. Almost all load generators have this problem.

We can look at a common load-testing example to see how this problem manifests. With this type of test, a client generally issues requests at a certain rate, measures the response time for each request, and puts them in buckets from which we can study percentiles later.

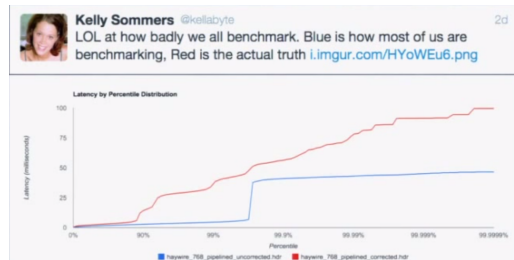
The problem is what if the thing being measured took longer than the time it would have taken before sending the next thing? What if you’re sending something every second, but this particular thing took 1.5 seconds? You wait before you send the next one, but by doing this, you avoided measuring something when the system was problematic. You’ve *coordinated* with it by backing off and not measuring when things were bad. To remain accurate, this method of measuring only works if all responses fit within an expected interval.

Coordinated omission also occurs in monitoring code. The way we typically measure something is by recording the time before, running the thing, then recording the time after and looking at the delta. We put the deltas in stats buckets and calculate percentiles from that. The code below is taken from a Cassandra benchmark.

```
/**
 * Performs the actual reading of a row out of the StorageService, fetching
 * a specific set of column names from a given column family.
 */
public static List<Row> read(List<ReadCommand> commands, ConsistencyLevel consistency_level)
    throws UnavailableException, IsBootstrappingException, ReadTimeoutException
{
    if (StorageService.instance.isBootstrappingMode())
        throw new IsBootstrappingException();
    long startTime = System.nanoTime();
    List<Row> rows;
    try
    {
        rows = fetchRows(commands, consistency_level);
    }
    finally
    {
        readMetrics.addNano(System.nanoTime() - startTime);
    }
    return rows;
}
```

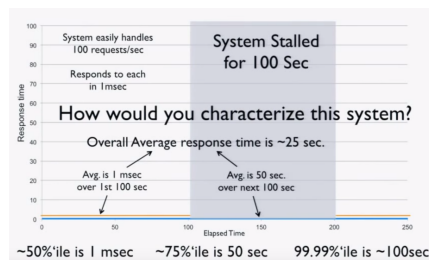
However, if the system experiences one of the “hiccups” described earlier, you will only have *one* bad operation and *10,000* other operations waiting in line. When those 10,000 other things go through, they will *look* really good when in reality the experience was *really bad*. Long operations only get measured once, and delays outside the timing window don’t get measured at all.

In both of these examples, we’re omitting data that looks bad on a very selective basis, but just how much of an impact can this have on benchmark results? It turns out the impact is *huge*.



Imagine a “perfect” system which processes 100 requests/second at exactly 1 ms per request. Now consider what happens when we freeze the system (for example, using CTRL+Z) after 100 seconds of perfect operation for 100 seconds and repeat. We can intuitively characterize this system:

- The average over the first 100 seconds is 1 ms.
- The average over the next 100 seconds is 50 seconds.
- The average over the 200 seconds is 25 seconds.
- The 50th percentile is 1 ms.
- The 75th percentile is 50 seconds.
- The 99.99th percentile is 100 seconds.



Subscribe to our newsletter

What I Learned About Billionaires at Jeff Bezos’s Private Retreat

NOAH HAWLEY · 02 MAY 2026 · [SOURCE](#)

For the richest men on Earth, everything is free and nothing matters.



(Illustration by Tim Enthoven)

At the end of Paul Thomas Anderson’s 2007 movie, *There Will Be Blood*, Daniel Day-Lewis’s oil-baron character, old now and richer than Croesus, beats Paul Dano’s preacher to death with a bowling pin. Dano’s Eli Sunday, a nemesis of Day-Lewis’s Daniel Plainview during his seminal, wealth-building years, has come to sell Plainview the oil-rich land that he once coveted. But Plainview doesn’t need the land anymore, because—as he explains in one of the most famous monologues in modern cinema—he has sucked out all the oil hidden beneath it from an adjoining property, like a milkshake.

Desperate for money, Eli begs for a loan. Instead, Plainview chases him around a bowling alley and murders him with great enthusiasm. Once it’s over, a butler comes to see what all the noise was about. “I’m finished,” Plainview yells.

No matter how many times I watch that movie, and I watch it a lot, I have never once taken those words to mean *I’m done for*. *There will now be consequences for my actions*. Quite the opposite: They mean that Plainview has completed his journey, through the acquisition of wealth and power, to a realm outside the moral universe. He’s finished, in other words, pretending that the rules of human society apply to him.

Buzzwords rise and fall. Try to catch them on the rise — go harder than everyone else on the rise up and regularly test their effectiveness, such that by the time your main buzzword has tapped out you’ve got 2-3 others cooking.

Workshopping your titles

I write this post out of frustration, but also to capture my own thought process. I’m currently procrastinating while writing a Latent Space post.

- The original title was Why every AI Engineer needs an AI Gateway. Good, genuine opinion, load bearing. But not super memorable.
- My next iteration was “**Stop writing model routers, use an LLM Gateway**”. Ok, prescriptive, evocative. But didn’t cover the other elements other than model routing. One could shorten to “Stop writing your own LLM Gateway” but that doesn’t quite express what people do.
- My next one is “**It’s not your job to DIY an LLM Gateway**”. No buzzword bc “your job” is implied (for a Latent Space audience), a little curiosity gap, LLM buzzword, strong opinion. But still not strong enough.
- Final one is “**LLM Gateway: The One Decision That Removes 100 AI Engineering Decisions**”. Leans on Tim Ferriss’ madlbs.

It’s very common for the “tail to wag the dog to wag the tail” - the title is so important to clickthrough and so impactful to the message, that you spend a bunch of time on just the title itself, because that determines the piece you will write, then write the piece, then you go back and tweak the title based on what happened.

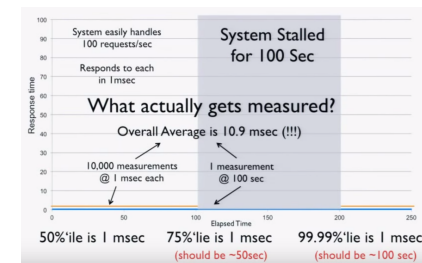
Amateur writers think their content has 95% of the value and then spend 5% on the title. Pros will put the value of titles closer to 50% and then act like it.

You Should Break The Rules Once You Understand Human Behavior More Intuitively Than Rules Can Model

See title of this post. No better aura than intentional, conspicuous, successful rulebreaking.

Now we try measuring the system using a load generator. Before freezing, we run 100 seconds at 100 requests/second for a total of 10,000 requests at 1 ms each. After the stall, we get one result of 100 seconds. This is the entirety of our data, and when we do the math, we get these results:

- The average over the 200 seconds is 10.9 ms (should be 25 seconds).
- The 50th percentile is 1 ms.
- The 75th percentile is 1 ms (should be 50 seconds).
- The 99.99th percentile is 1 ms (should be 100 seconds).



Basically, your load generator and monitoring code tell you the system is ready for production, when in fact it’s lying to you! A simple “CTRL+Z” test can catch coordinated omission, but people rarely do it. It’s critical to calibrate your system this way. If you find it giving you these kind of results, throw away all the numbers—they’re worthless.

You have to measure at random or “fair” rates. If you measure 10,000 things in the first 100 seconds, you have to measure 10,000 things in the second 100 seconds during the stall. If you do this, you’ll get the correct numbers, but they won’t be as pretty. Coordinated omission is the simple act of erasing, ignoring, or missing all the “bad” stuff, but the data is good.

Surely this data can still be useful though, even if it doesn’t accurately represent the system? For example, we can still use it to identify performance regressions or validate improvements, right? Sadly, this couldn’t be further from the truth. To see why, imagine we improve our system. Instead of pausing for 100 seconds after 100 seconds of perfect operation, it handles all requests at 5 ms each after 100 seconds. Doing the math, we get the following:

- The 50th percentile is 1 ms
- The 75th percentile is 2.5 ms (stall showed 1 ms)

- The 99.99th percentile is 5 ms (stall showed 1 ms)

This data tells us we *hurt* the four nines and made the system 5x *worse*! This would tell us to revert the change and go back to the way it was before, which is clearly the *wrong* decision. With bad data, *better can look worse*. This shows that you cannot have any intuition based on any of these numbers. The data is garbage.

With many load generators, the situation is actually much *worse* than this. These systems work by generating a constant load. If our test is generating 100 requests/second, we run 10,000 requests in the first 100 seconds. When we stall, we process just one request. After the stall, the load generator sees that it's 9,999 requests behind and issues those requests to catch back up. Not only did it get rid of the *bad* requests, it replaced them with *good* requests. Now the data is *twice* as wrong as just dropping the bad requests.

What coordinated omission is really showing you is *service time*, not response time. If we imagine a cashier ringing up customers, the *service time* is the time it takes the cashier to do the work. The *response time* is the time a customer waits before they reach the register. If the rate of arrival is higher than the service rate, the response time will continue to grow. Because hiccups and other phenomena happen, response times often bounce around. However, coordinated omission lies to you about response time by actually telling you the service time and hiding the fact that things stalled or waited in line.

Measuring Latency

Latency doesn't live in a vacuum. Measuring response time is important, but you need to look at it in the context of load. But how do we properly measure this? When you're nearly idle, things are nearly perfect, so obviously that's not very useful. When you're pedal to the metal, things fall apart. This is *somewhat* useful because it tells us how "fast" we can go before we start getting angry phone calls.

However, studying the behavior of latency at saturation is like looking at the shape of your car's bumper after wrapping it around a pole. The only thing that matters when you hit the pole is that *you hit the pole*. There's no point in trying to engineer a better bumper, but we can engineer for the speed at which we lose control. Everything is going to suck at saturation, so it's not super useful to look at beyond determining your operating range.

- opposite also works:

- **The Rise of {THING}**

- **The Unreasonable Effectiveness of {THING}**

- **Fundamentals of {THING}**

- straightforward, people love fundamentals

- also **How {THING} Really Works**

- You can also flip it and go for the **Advanced** {THING} tier content

- {CONVEY EXPERIENCE/SCALE} with {THING}

- e.g. What We Learned from a Year of Building with LLMs, Insights from over **10,000** comments on "Ask HN: Who Is Hiring" using GPT-4o

There's loads more templates that work, please suggest in comments and I can add/review.

You can also browse the masters of microcopy like Steph Smith and Shaan Puri, and if you are worried about clickbait, you can see Veritasium's take or Simon Willison's Highlights.

also Andrew Chen senpai:

Understand Buzzword Metagame

Buzzwords work. Because of a mix of human psychology and algorithmic recsys. Don't avoid them, just use them judiciously. This involves the ability to read the room, which many smart people sadly cannot without significant involvement in the broader community you are trying to speak to. I don't see any way around it other than trying to authentically immerse, or consult a friendly who does.

Not all buzzwords are created equal. Databricks/Berkeley tried to push Compound AI Systems, Gartner is pushing Composite AI and Composable business — this resonates with their audience, but not more broadly.

Pick the shibboleths that work for the community you want. AI Engineer worked for reasons that are well understood now. GraphRAG is working outside of the company that coined it. These buzzwords travel, go ahead and put them in there.

Curiosity Gap Dynamics

Marketers know the power of [the Curiosity Gap](#). Here is where we toy closest with clickbait. I'll be honest that I wasn't even sure if I wanted to give it its own subheading, but its a very important concept that you should at least be aware exists, even if you're going to ignore it.

Strong enough opinions create their own curiosity gaps; you won't have to try very hard to make it interesting if you got my attention in 4 words anyway.

Curiosity gaps done poorly can lead to "[burying the lede](#)", which make your talk less Quotable and you also don't want to do that.

Simplest way to put it is; if the most interesting thing about your content is your title, and what people take away after having read your post matches what they would have predicted after reading your title, then you've probably said too much and could cut.

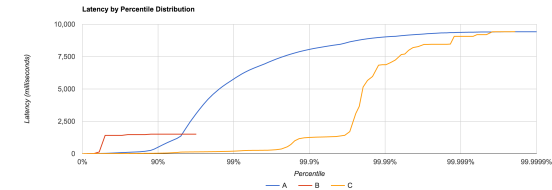
Play Madlibs

You don't have to invent your title from scratch (nor should you always do what I'm about to suggest). Here's some templates that work well:

- {*BENEFIT EVERYONE WANTS*} **without** {*DOWNSIDE EVERYBODY EXPECTS*}
 - e.g. [How to Get Rich \(Without Getting Lucky\)](#), [How to Market Yourself \(Without Being A Celebrity\)](#). See also subtitle of this post.
- {*PRESCRIPTIVE OPINION*}, **don't** {*THING YOU DO*}
 - e.g. [Parse, don't validate](#)
 - alternative prescriptive opinion: **Never** {*THING YOU DO*}
 - see also title of this post
- {*POPULAR THING*} **IS DEAD**
 - unfortunately this is clickbait that works
 - e.g. [Prompt Engineering is Dead](#), [Big Data is Dead](#)
 - when given at a conference for *POPULAR THING*, this is known as a [Heresy Talk](#)

What's more important is testing the speeds in between idle and hitting the pole. Define your SLAs and plot those requirements, then run different scenarios using different loads and different configurations. This tells us if we're meeting our SLAs but also how many machines we need to provision to do so. If you don't do this, you don't know how many machines you need.

How do we capture this data? In an ideal world, we could store information for every request, but this usually isn't practical. [HdrHistogram](#) is a tool which allows you to capture latency and retain high resolution. It also includes facilities for correcting coordinated omission and plotting latency distributions. The original version of HdrHistogram was written in Java, but there are versions for many other languages.



To Summarize

To understand latency, you *have* to consider the entire distribution. Do this by plotting the latency distribution curve. Simply looking at the 95th or even 99th percentile is not sufficient. Tail latency matters. Worse yet, the median is *not* representative of the “common” case, the average even less so. There is no single metric which defines the behavior of latency. Be conscious of your monitoring and benchmarking tools and the data they report. You can't average percentiles.

Remember that *latency is not service time*. If you plot your data with coordinated omission, there's often a quick, high rise in the curve. Run a “CTRL+Z” test to see if you have this problem. A non-omitted test has a much smoother curve. Very few tools actually correct for coordinated omission.

Latency needs to be measured in the context of load, but constantly running your car into a pole in every test is not useful. This isn't how you're running in production, and if it is, you probably need to provision more machines. Use it to establish your limits and test the sustainable throughputs in between to determine if you're meeting your SLAs. There are a lot of flawed tools out there, but HdrHistogram is one of the few that isn't. It's useful for benchmarking and, since histograms are additive and HdrHistogram uses log buckets, it can also be useful for capturing high-volume data in production.

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 30 MAR 2026 · [SOURCE](#)

People code switch. Every ethnic minority has [lived experience of this](#). When you talk to me in Singapore, I sound very different than when I'm in San Francisco. And the way you talk at work is probably very different from how you talk to your kids and how you talk to your lover.

I've been dealing with a particularly virulent form of code switching in the simple form of titling blogposts and talks. When you ask the writer face to face what their thing is about, they can give you a perfectly human explanation. And then when they send you their piece, it's all *“Leveraging GenAI with Robust Data Foundations to Transform Businesses”*. I've left my body by the time I read the first word.

My hope in writing this piece is to **put an end to bad titles**.

If I sent you this guide, it is not an insult. You just don't have good title game. That's ok. You're great at what you do, title game isn't your job. I'm the guy that wrote [How To Name Things](#) and [Two Words](#). I sent you this guide *because* your shitty title is the main thing stopping your readers from getting to the good part. If you didn't have anything good, I wouldn't bother with a lost cause.

First...

Towards Leveraging Corporate Speak For Maximum Enterprise Synergy

DO NOT:

- Start with “Leveraging” or “Towards” or “Navigating” or “Empowering”

Yeah just stop. There's no rescuing this when you start there. Just delete meaningless corporate weasel words. Anything >3 syllables is a red flag.

Make It Quotable

If you do not try to make your title roll off the tongue, it probably won't.

Literally imagine telling your friend about your talk. Telling a new acquaintance to google your blogpost. Having a fan of yours cite it back to you. How much do they stumble? Yeah. This has all happened to me.

There's an extremely high correlation between what are considered the best talks and the quotability of titles. [Simple Made Easy](#). [Inventing on Principle](#). Our list of best [demos/pitches](#) of all time.

Resist colons. It lacks confidence. Why Use Lot Word: ~~When Few Word Do Trick~~? You're not [High School Musical: The Musical: The Series](#).

Put the SEO keyword vomit in the description.

Titles are for humans.

Prescribe A Strong Opinion

Opinions are one of the most quotable things.

But more importantly, a good Opinion Piece is load bearing because it is eminently forwardable, and therefore viral and impactful. Your Job To Be Done is to be the definitive, conclusive, piece on that Opinion. Stake out territory in Opinion Space, list out all the arguments for, counter all the arguments against, provide datapoints, and watch that go further than anything you've ever done.

How to know what's a good opinion? **Three Strikes rule:** If you repeat yourself [three times](#), ship it. Even if you're just quoting others, its fine, credit them and make it your own.

Mental Models and Frameworks are more complex opinions that allow others to scaffold their thought around you. My [Radiating Circles](#) and [Measuring Developer Relations](#) and [Third Age of JavaScript](#) did this.

Opinions contrary to an established opinion are the hardest to do well and the best when correct. [What Clayton Christensen Got Wrong](#) made Ben Thompson's career. [Rise of AI Engineer](#) countered the prevailing Prompt Engineer -and- GPU Rich Startup/ML Engineer narrative at the time, with room to grow.